

Informe Técnico de Arquitectura: Plataforma Social

Docente:

Claudio Esteban Rojas Rodriguez

Integrantes:

Santiago Catalan

Nelson Baeza

Bruno Stockle

Sección: 003D

1. Introducción y Contexto del Sistema

1.1 introducción / Contexto

El proyecto Mazanex es una plataforma que combina una interfaz de usuario Next.js con un backend compuesto por varios microservicios Java Spring Boot. El frontend se comunica a través de un BFF basado en KrakenD y los microservicios se ejecutan en contenedores Docker junto con una base de datos MySQL. En el repositorio el sistema está organizado de esta manera:

- frontend para la aplicación web Next.js
- bff para el gateway KrakenD
- ms-auth, ms-profile, ms-publications, ms-ranking, ms-projects para los microservicios
- docker-compose.yml para orquestar todos los contenedores
- migraciones automatizadas con Flyway

1.2 Propósito del Proyecto

Proveer servicios de comunidad y colaboración: registro/autenticación, perfiles, feed de publicaciones (posts/likes/comentarios), almacenamiento de puntuaciones de juegos y gestión de tareas/proyectos. El requisito no funcional principal es facilitar despliegues reproducibles en contenedores y asegurar autenticación basada en JWT.

1.3 Alcance Tecnológico

- **Frontend:** Next.js (React + TypeScript), estilos con Tailwind; código en frontend.
- **Gateway / BFF:** KrakenD con rutas definidas en krakend.json.
- **Backend:** microservicios Java 25 + Spring Boot en ms-auth, ms-profile, ms-publications, ms-ranking, ms-projects.
- **Persistencia:** MySQL 8. Para la gestión del ciclo de vida y la evolución del esquema de base de datos, se utiliza Flyway como herramienta de migración.
- **Seguridad y auth:** JWT emitido por ms-auth y validado localmente por JwtAuthenticationFilter en cada servicio.
- **Contenerización:** docker-compose.yml para orquestación local; Dockerfile en cada microservicio y manifiestos k8s/ en algunos módulos.
- **Integración interna:** REST síncrono entre servicios (actualmente RestTemplate).

2. Decisiones Arquitectónicas Críticas

Se priorizó modularidad y despliegue independiente mediante microservicios. KrakenD actúa como BFF para centralizar ruteo, CORS y políticas de borde. Se eligió validación JWT local para reducir dependencia de llamadas remotas de autorización.

2.1 Microservicios vs Arquitectura Monolítica

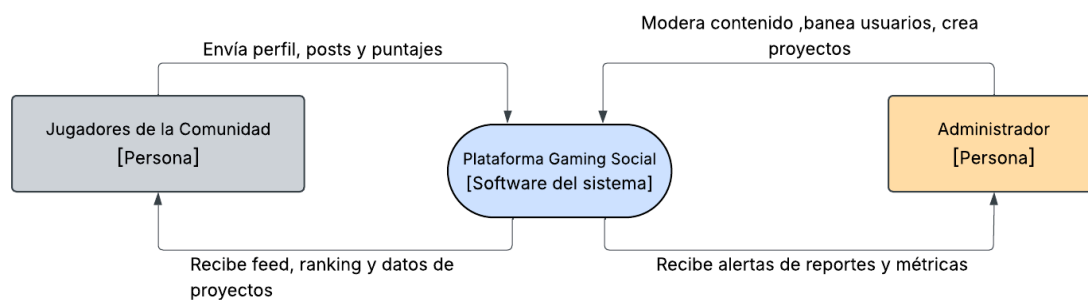
- Separación de dominios (auth, profile, publications, ranking, projects) para facilitar responsabilidad única, despliegue independiente y escalado dirigido.
- Riesgos: mayor complejidad operativa, necesidad de observabilidad distribuida, coordinación de versiones y mayor carga de pruebas de integración

2.2 Seguridad Descentralizada (JWT Local)

- ms-auth genera tokens JWT usando el servicio JwtService.
- Cada microservicio backend tiene un filtro JwtAuthenticationFilter y una clase SecurityConfig.
- En SecurityConfig.java, se permite el acceso a /api/auth/register y /api/auth/login.
- En SecurityConfig.java, se permite acceso público a /api/profile/list, /api/profile/social/public/** y /api/profile/sync.
- En SecurityConfig.java, se protege /api/projects/**.

3. Diagrama de contexto

El Diagrama de Contexto (Nivel 1) establece los límites de la plataforma, abstrayendo por completo la infraestructura técnica subyacente para enfocarse en el valor del negocio y la interacción directa con los usuarios.



3.1 Descripción de Interacciones

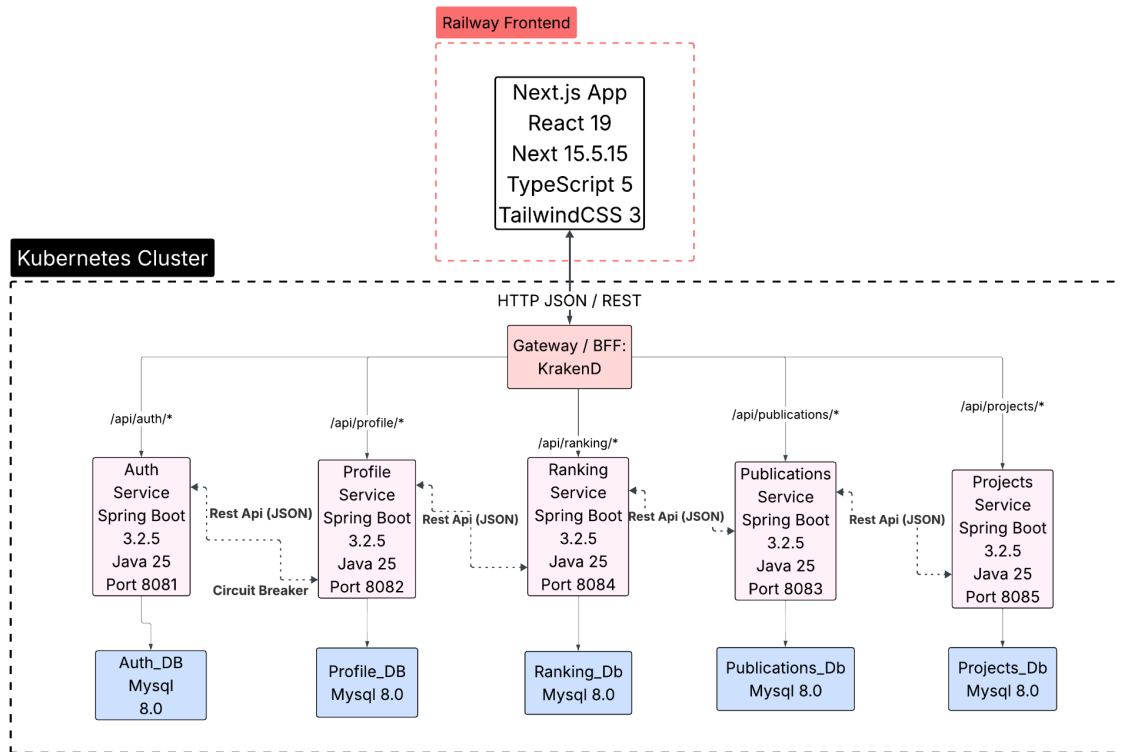
- **Jugadores de la Comunidad:** Utilizan la plataforma como un punto de encuentro centralizado donde acceden a servicios de autenticación, gestionan su identidad digital (perfiles), comparten contenido social en el feed, y visualizan tanto su rendimiento competitivo en el ranking global como su progreso en proyectos colaborativos.
- **Administrador:** Actúa como el garante de la integridad y seguridad de la comunidad, con privilegios exclusivos para la creación y gestión de proyectos, además de herramientas de moderación de contenido y baneo de usuarios para asegurar un entorno saludable.

La plataforma actúa como un facilitador que unifica estos servicios, garantizando una experiencia de usuario fluida y segura independientemente de la infraestructura técnica subyacente.

4. Arquitectura del Sistema - C2(Contenedores)

Este nivel describe las unidades ejecutables de despliegue y su topología de red, aislando el comportamiento de las aplicaciones.

El contenedor BFF expone el puerto 8080. El contenedor frontend expone el puerto 3000. Cada microservicio expone su propio puerto interno mapeado a la máquina



4.1 Descripción de Contenedores y Flujos

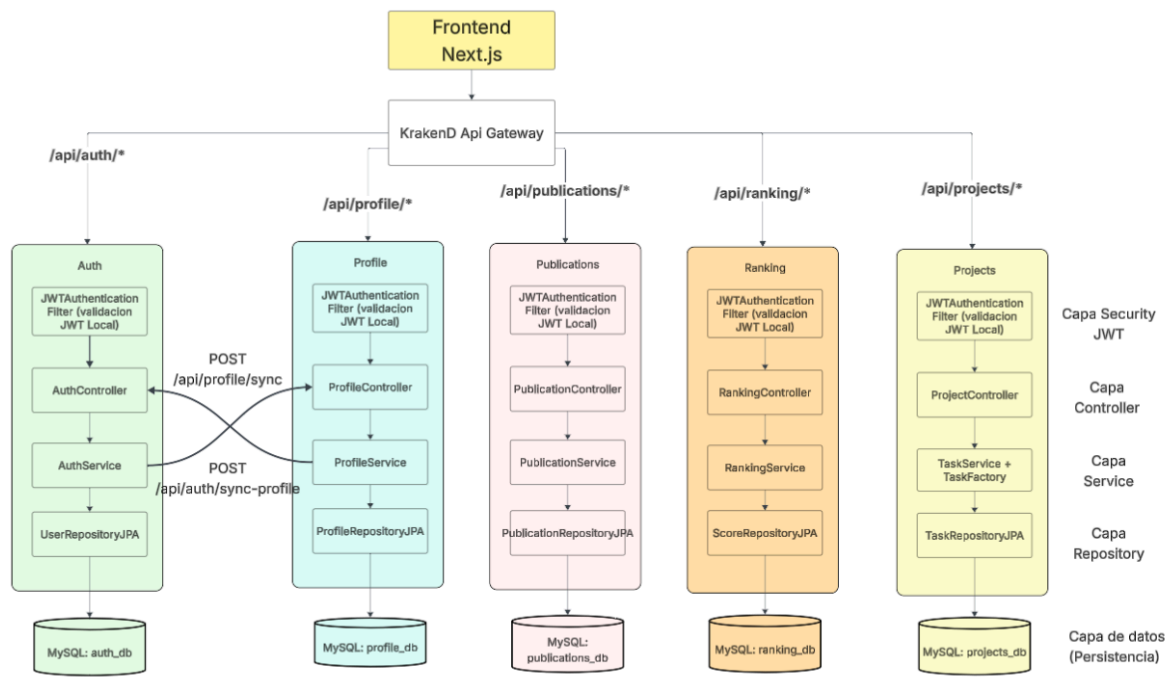
El sistema opera mediante cuatro componentes clave comunicados por protocolos estándar:

- **Frontend (Next.js):** Interfaz visual que consume los servicios de backend mediante peticiones HTTP/JSON.
- **Gateway / BFF (KrakenD):** Único punto de entrada público que resuelve CORS y enruta el tráfico web hacia las rutas internas.
- **Microservicios (Spring Boot):** Clúster de cinco servicios independientes; cada uno opera su propia API REST aislada en un puerto interno específico con la lógica de su dominio.
- **Persistencia (MySQL 8):** Bases de datos lógicas aisladas por microservicio para garantizar el desacoplamiento de la información.

4.2 Topología de Despliegue

En el entorno de desarrollo, el sistema se orquesta mediante Docker Compose, permitiendo levantar todo el ecosistema con un solo comando. Para el paso a producción, la arquitectura está diseñada para ser migrada a un clúster de Kubernetes, implementando *Ingress Controllers*, inyección de secretos mediante *Secret Manager*, y políticas de Autoescalado Horizontal de Pods (HPA) apoyadas en *probes* de disponibilidad (liveness/readiness).

5. Arquitectura del Sistema - C3 (Componentes)



5.1 Flujo Interno de Microservicios

Flujo estándar dentro de un microservicio:

Petición HTTP → filtro JWT (JwtAuthenticationFilter) → controlador REST → servicio de negocio → repositorio JPA → base de datos MySQL.

En ms-auth, además existe un flujo de sincronización con ms-profile usando RestTemplate dentro de AuthService.syncWithProfile.

5.2 Patrones de Diseño y Estructura Aplicados

La plataforma se fundamenta en los siguientes patrones clave para asegurar su mantenibilidad:

- **BFF / API Gateway (Patrón Arquitectural):** KrakenD consolida las peticiones del cliente y oculta la topología interna del clúster.
- **Arquitectura en Capas (Patrón Estructural):** Separación estricta en capas lógicas de Presentación, Negocio y Persistencia.
- **Intercepting Filter (Patrón de Seguridad):** El `JwtAuthenticationFilter` valida credenciales sin contaminar la lógica de los controladores.
- **Factory Method (Patrón Creacional):** Implementado en el módulo `ms-projects` a través de la clase `TaskFactory`. Este patrón aísla la complejidad de instanciación de nuevas entidades, garantizando que cada tarea se cree con un estado inicial inmutable y válido.
- **Repository Pattern:** Uso de interfaces JPA para abstraer las operaciones SQL.
- **Synchronous Service Calls:** Uso de `RestTemplate` para orquestar la sincronización de datos entre dominios en tiempo real.
- **Documentación de API (OpenAPI/Swagger):** Implementación de *SpringDoc OpenAPI* en cada microservicio para generar contratos de API dinámicos. Esto proporciona al equipo de frontend una fuente de verdad única y actualizada sobre los *endpoints* disponibles, facilitando la integración y reduciendo errores en el consumo de los servicios.

Links Swagger

Auth Service (ms-auth): <http://localhost:8081/swagger-ui/index.html>

Profile Service (ms-profile): <http://localhost:8082/swagger-ui/index.html>

Publications Service (ms-publications): <http://localhost:8083/swagger-ui/index.html>

Ranking Service (ms-ranking): <http://localhost:8084/swagger-ui/index.html>

Projects Service (ms-projects): <http://localhost:8085/swagger-ui/index.html>

6.Evidencia de Implementación y Aseguramiento de Calidad

Para respaldar las decisiones arquitectónicas detalladas en los apartados anteriores, esta sección expone las evidencias prácticas de la construcción del sistema, abarcando desde la seguridad en el cliente hasta la infraestructura de despliegue y la cobertura de pruebas.

6.1 Interfaz de Usuario y Experiencia (Frontend)

Para materializar la interacción del usuario final con el ecosistema de microservicios, se construyó una aplicación web robusta utilizando Next.js (React) y TypeScript. El diseño de la interfaz gráfica se implementó con Tailwind CSS, garantizando una experiencia de usuario (UX) fluida, moderna y responsiva en distintos dispositivos.

El frontend actúa como el principal consumidor del API Gateway (KrakenD). A través del uso de herramientas como Axios para las peticiones HTTP y Zustand para el manejo del estado global, la aplicación logra orquestar eficientemente la información proveniente de los distintos dominios (autenticación, perfiles, publicaciones, ranking y proyectos) presentándola en una interfaz unificada.

Crear cuenta

Completa los datos para registrarte

NOMBRE

Tu nombre

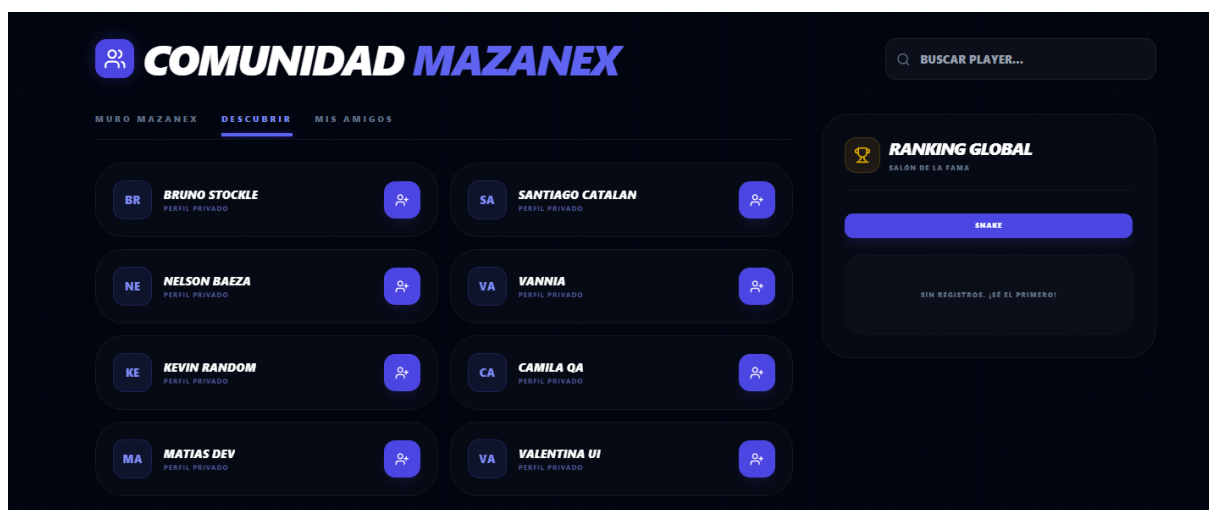
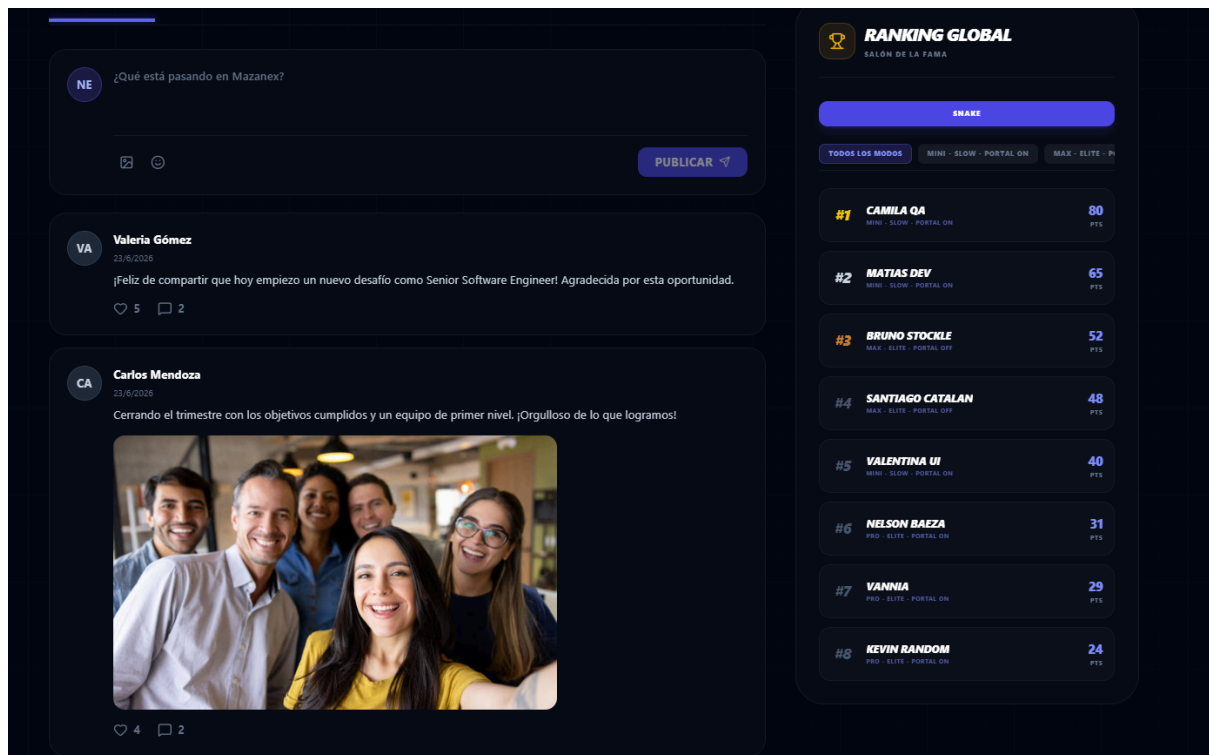
CORREO

correo@mail.com

CONTRASEÑA

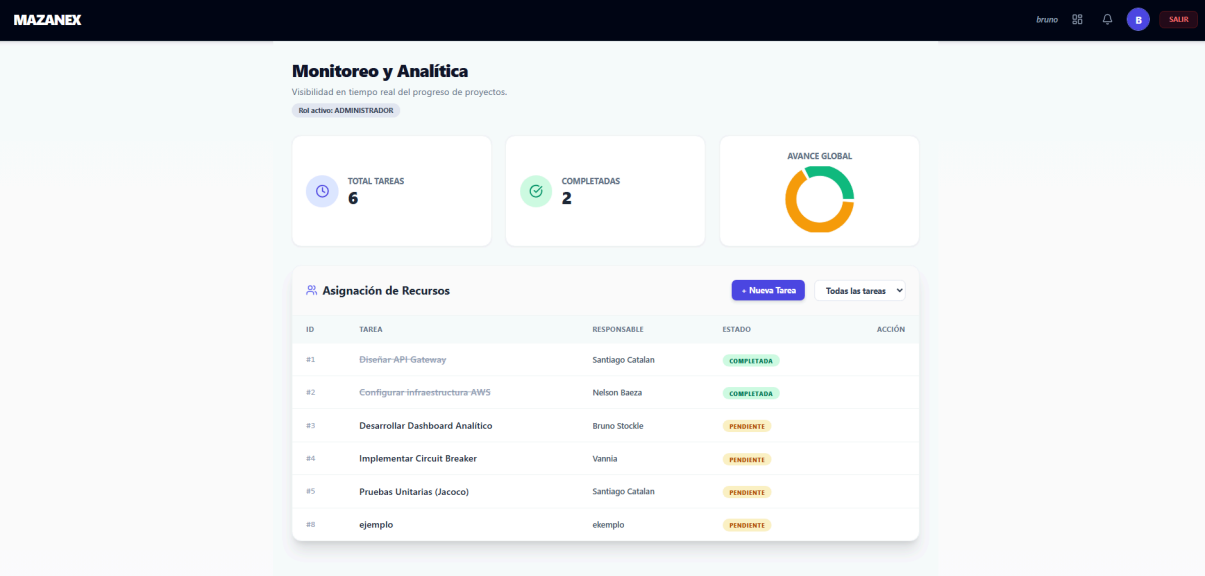
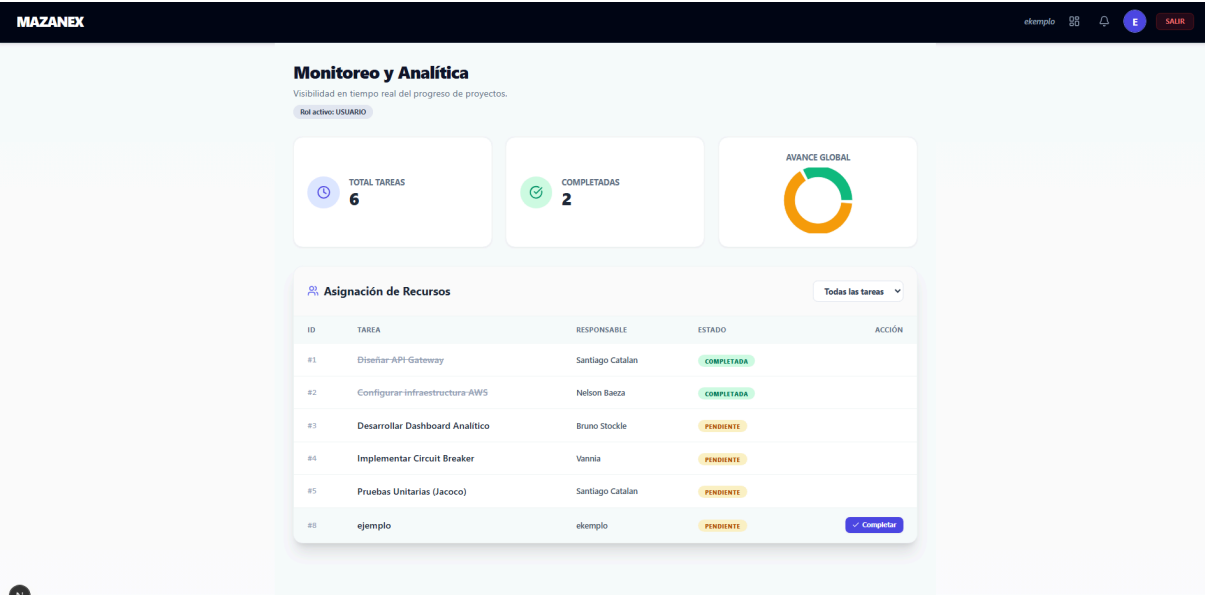
.....

Registrarse



6.2 Vistas Condicionales y Seguridad en el Cliente (JWT)

Para validar la integración de la seguridad descentralizada, el frontend (Next.js) implementa la decodificación del payload del token JWT, permitiendo renderizar la interfaz de manera condicional según el rol del usuario autenticado. Como se observa en las capturas del panel de monitoreo, un perfil con rol ADMINISTRADOR cuenta con acceso a controles de creación (como el botón "+ Nueva Tarea"), mientras que un USUARIO estándar visualiza una interfaz restringida, enfocada exclusivamente en la visualización y ejecución de tareas asignadas.



6.3 Infraestructura Preparada para Producción (Kubernetes)

Si bien el entorno de desarrollo local se orquesta eficientemente mediante Docker Compose, el repositorio ya incorpora la Infraestructura como Código (IaC) requerida para la migración a un entorno de producción. En microservicios críticos como ms-auth, se han estructurado los manifiestos esenciales para un clúster de Kubernetes (K8s), incluyendo `deployment.yaml` para la gestión de réplicas, `secret.yaml` para credenciales y `hpa.yaml` para habilitar el Autoescalado Horizontal de Pods (HPA).

Gordon

Containers

Images

Logs

Volumes

Kubernetes

Builds

Models

Docker Hub

MCP Toolkit

Extensions

Containers

Give feedback

Container CPU usage3.27% / 400% (4 CPUs available)

Container memory usage2.51GB / 7.56GB

Show charts

Search

Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Memory usag...	Memo	Actions
	mazanex	-	-	-	3.27%	2.51GB / 61.91Gi		
	db	6832d7eb99e6	mysql:8.0	3306:3306	0.79%	498.9MB / 7.74G		
	ranking	96b128cdf17c	mazanex-ranking-service	8084:8084	0.15%	230.6MB / 7.74G		
	profile	d2b3270f5b5b	mazanex-profile-service	8082:8082	0.16%	231.2MB / 7.74G		
	auth	be419e6e3353	mazanex-auth-service	8081:8081	0.13%	241.5MB / 7.74G		
	projects	a2069a8cda2f	mazanex-projects-service	8085:8085	1.9%	241.5MB / 7.74G		
	publications	68113fcd500	mazanex-publications-serv	8083:8083	0.14%	223.4MB / 7.74G		
	bff	6f904da3743e	mazanex-bff	8080:8080	0%	33.67MB / 7.74G		
	frontend	1a682bb75e6d	mazanex-frontend	3000:3000	0%	874.1MB / 7.74G	1	

Upgrade plan

Engine runningKubernetes running RAM 7.48 GB CPU 5.50% Disk: 19.69 GB used (limit 1006.85 GB)Terminal v4.78.0

ms-auth

.mvn

k8s

configmap.yaml

deployment.yaml

hpa.yaml

kustomization.yaml

pdb.yaml

secret.yaml

src

target

classes

generated-sources

generated-test-sources

maven-status

site

surefire-reports

test-classes

jacoco.exec

Dockerfile

mvnw

mvnw.cmd

pom.xml

3

README.md

6.4 Aseguramiento de Calidad y Testing Automatizado

Para prevenir regresiones y garantizar la estabilidad del ecosistema, se implementó una suite de pruebas unitarias con un enfoque estratégico. En la arquitectura propuesta (Spring Boot), el core del sistema y todo el funcionamiento transaccional residen exclusivamente en la Capa de Servicios (Service Layer). Por lo tanto, los esfuerzos de testing se concentraron en validar exhaustivamente esta capa, priorizando la lógica de negocio por sobre el código repetitivo (como configuraciones, DTOs o controladores pasivos).

← → ↻ file:///c%3A/Users/Bruno/OneDrive/Documentos/GitHub/mazanex/backend/ms-auth/target/site/jacoco/index.html

auth

auth

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.mazanex.auth.config		0 %		0 %	29	29	66	66	21	21	4	4
com.mazanex.auth.service		67 %		39 %	12	25	16	56	2	11	0	1
com.mazanex.auth.controller		0 %		0 %	10	10	20	20	7	7	1	1
com.mazanex.auth.model		69 %		n/a	5	19	5	19	5	19	0	1
com.mazanex.auth.dto		69 %		n/a	5	7	7	9	5	7	1	3
com.mazanex.auth		0 %		n/a	3	3	4	4	3	3	1	1
Total	543 of 808	32 %	39 of 50	22 %	64	93	118	174	43	68	7	11

← → ↻ file:///c%3A/Users/Bruno/OneDrive/Documentos/GitHub/mazanex/backend/ms-profile/target/site/jacoco/index.html

perfil

perfil

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.mazanex.profile.model		16 %		3 %	110	134	30	62	41	65	0	4
com.mazanex.profile.config		0 %		0 %	28	28	72	72	20	20	4	4
com.mazanex.profile.service		65 %		50 %	21	46	32	102	6	27	0	2
com.mazanex.profile.controller		0 %		0 %	17	17	24	24	15	15	2	2
com.mazanex.profile		0 %		n/a	3	3	4	4	3	3	1	1
Total	1.406 of 1.928	27 %	172 of 196	12 %	179	228	162	264	85	130	7	13

file:///c%3A/Users/Bruno/OneDrive/Documentos/GitHub/mazanex/backend/ms-publications/target/site/jacoco/index.html

publications

publications

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.mazanex.publications.model		12 %		1 %	97	117	13	28	27	46	0	2
com.mazanex.publications.dto		13 %		0 %	53	70	0	11	11	28	0	2
com.mazanex.publications.config		0 %		0 %	28	28	73	73	20	20	4	4
com.mazanex.publications.controller		0 %		n/a	7	7	33	33	7	7	1	1
com.mazanex.publications		0 %		n/a	3	3	4	4	3	3	1	1
com.mazanex.publications.service		93 %		100 %	2	11	0	33	2	10	0	1
Total	1.490 of 1.786	16 %	240 of 244	1 %	190	236	123	182	70	114	6	11

file:///c%3A/Users/Bruno/OneDrive/Documentos/GitHub/mazanex/backend/ms-ranking/target/site/jacoco/index.html

ranking

ranking

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.mazanex.ranking.model		10 %		0 %	68	72	11	24	25	29	0	1
com.mazanex.ranking.config		0 %		0 %	28	28	68	68	20	20	4	4
com.mazanex.ranking.dto		0 %		0 %	44	44	7	7	17	17	1	1
com.mazanex.ranking.controller		0 %		n/a	6	6	13	13	6	6	1	1
com.mazanex.ranking.service		67 %		58 %	6	12	10	30	2	6	0	1
com.mazanex.ranking		0 %		n/a	2	2	3	3	2	2	1	1
Total	1.129 of 1.275	11 %	161 of 168	4 %	154	164	112	145	72	80	7	9

file:///c%3A/Users/Bruno/OneDrive/Documentos/GitHub/mazanex/backend/ms-projects/target/site/jacoco/index.html

projects

projects

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.mazanex.projects.model		6 %		0 %	62	70	7	11	20	28	1	2
com.mazanex.projects.config		0 %		0 %	19	19	45	45	15	15	4	4
com.mazanex.projects.controller		0 %		0 %	6	6	10	10	5	5	1	1
com.mazanex.projects		0 %		n/a	3	3	4	4	3	3	1	1
com.mazanex.projects.factory		83 %		n/a	1	2	1	6	1	2	0	1
com.mazanex.projects.service		100 %		n/a	0	6	0	13	0	6	0	1
Total	631 of 727	13 %	94 of 94	0 %	91	106	67	89	44	59	7	10

Como se evidencia en los reportes de métricas generados por JaCoCo, el equipo superó con éxito el umbral del 60% de cobertura en los componentes críticos de todos los microservicios. Específicamente, los paquetes de servicios alcanzaron los siguientes niveles:

- ms-projects (com.mazanex.projects.service): 100% de cobertura.
- ms-publications (com.mazanex.publications.service): 93% de cobertura.
- ms-auth (com.mazanex.auth.service): 67% de cobertura.
- ms-ranking (com.mazanex.ranking.service): 67% de cobertura.
- ms-profile (com.mazanex.profile.service): 65% de cobertura.

Este enfoque selectivo asegura que los flujos principales de la plataforma estén sólidamente respaldados, garantizando la calidad del software al probar los componentes que realmente aportan valor al ecosistema.

7. Modelo de Datos y Sincronización

La estructura inicial de las bases de datos es gestionada por Flyway mediante las migraciones V1 de cada microservicio, asegurando la consistencia y evolución del esquema en:

- auth_db
- profile_db
- publications_db
- ranking_db
- projects_db.

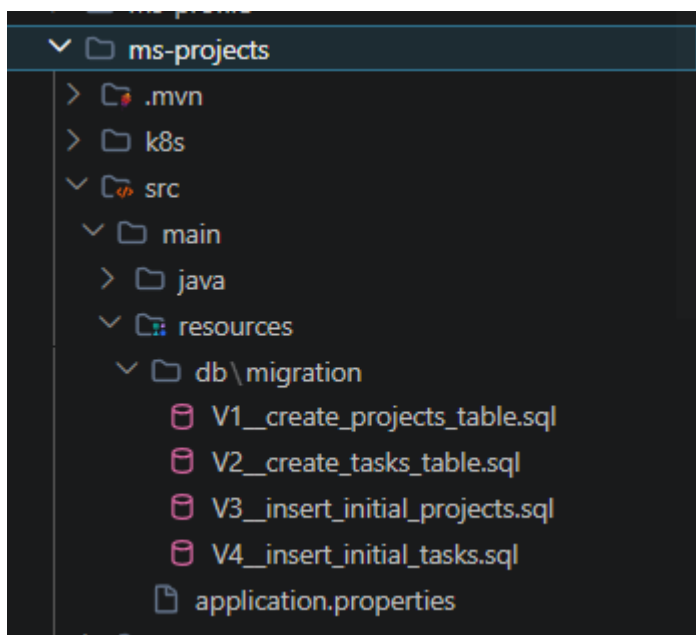
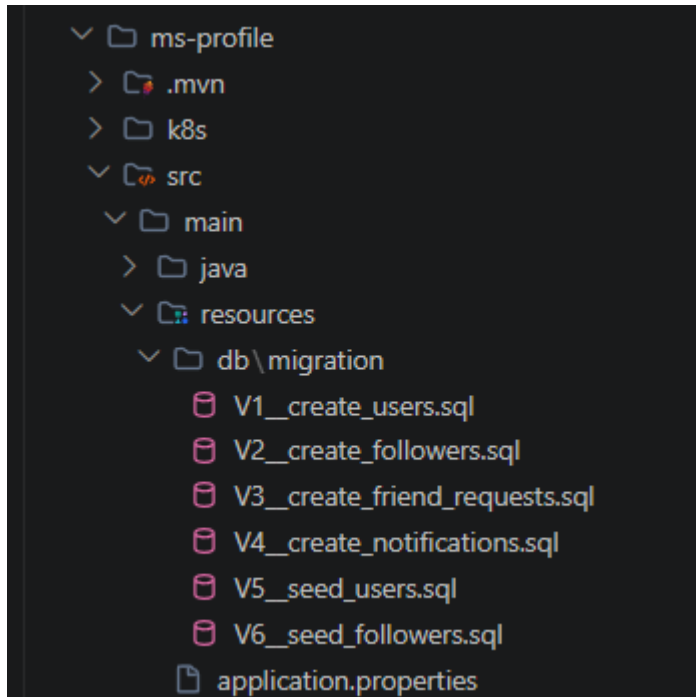
7.1 Aislamiento de la Persistencia

El ecosistema respeta la regla de propiedad de los datos (*ownership*). Cada microservicio se conecta de forma exclusiva a su propia base de datos lógica (auth_db, profile_db, publications_db, ranking_db, projects_db). Este aislamiento previene acoplamientos y facilita futuras migraciones.

Para asegurar la integridad y evolución de estas estructuras, se ha implementado Flyway. Este componente gestiona el ciclo de vida del esquema de base de datos de manera automatizada:

- **Migraciones Versionadas:** Cada cambio estructural se registra mediante archivos de migración numerados secuencialmente. Permitiendo una trazabilidad total de los cambios desde el despliegue inicial.

- **Automatización:** Al iniciar cada contenedor de microservicio, Flyway verifica automáticamente la versión actual del esquema y aplica las migraciones pendientes antes de que la aplicación reciba tráfico, garantizando que el sistema sea siempre compatible con la versión del código desplegado.
- **Consistencia de Entorno:** Flyway asegura que, independientemente del entorno (local o clúster de Kubernetes), la base de datos siempre alcance el estado definido por el código, evitando discrepancias en la estructura de datos.



7.2 Sincronización Interservicios (RestTemplate)

Para mantener la consistencia operativa, servicios como ms-auth utilizan RestTemplate para comunicarse de manera síncrona con otros dominios (por ejemplo, enviando datos de usuario al ms-profile tras un registro exitoso)

8.Consideraciones Éticas, Privacidad y Conclusión

8.1 Privacidad de Datos y Control de Acceso (RBAC)

El sistema asegura la autenticación mediante JWT y el modelo de base de datos ya incluye el atributo role (USER por defecto). Como trabajo futuro prioritario, se implementará autorización basada en roles (RBAC) a nivel de controladores usando Spring Security (ej. @PreAuthorize), habilitando de forma segura las capacidades de moderación del administrador.

8.2 Sincronización Interservicios y Resiliencia

La sincronización síncrona actual entre ms-auth y ms-profile (vía RestTemplate) está protegida por el patrón Circuit Breaker para gestionar latencias y evitar fallos en cascada. Para optimizar esto, se propone evolucionar hacia una Arquitectura Orientada a Eventos (RabbitMQ/Kafka) que reemplace esta comunicación y garantice un desacoplamiento total.

8.3 Conclusión Técnica

La arquitectura de microservicios (Spring Boot), Docker y KrakenD Gateway demuestra ser una base sólida y escalable. La separación de bases de datos y el uso de patrones (Factory Method, Repository, Circuit Breaker) aportan gran madurez al código. Para consolidar el paso a producción (ej. Kubernetes), las prioridades estratégicas son saldar la deuda técnica: aplicar el RBAC y migrar a una comunicación interna completamente asíncrona.